МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу
«Информационный поиск»

Лемматизация

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 22.10.2026
Оценка:

Москва, 2026

Содержание

1 Постановка задачи 3

2 Метод решения 3

2.1 Корпус и словарные данные . . . 3

2.2 Собственный анализатор на C++ . . . 3

2.3 Сравнение поиска до и после . . . 4

2.4 Тестирование и измерения . . . 5

3 Результаты 5

3.1 Примеры соответствия токенов и лемм . . . 5

3.2 Обработка корпуса . . . 7

3.3 Качество и ошибки поиска . . . 8

3.4 Исходный код и воспроизводимость . . . 9

4 Вывод 9

1 Постановка задачи

Добавить в созданную поисковую систему лемматизацию. В простейшем случае, это просто поиск без учёта словоформ. В более сложном случае, можно давать бонус большего размера за точное совпадение слов.

Лемматизацию можно добавлять на этапе индексации, можно на этапе выполнения поискового запроса.

В отчёте должна быть включена оценка качества поиска, после внедрения лемматизации. Стало ли лучше? Изучите запросы, где качество ухудшилось. Объясните причину ухудшения и как можно было бы улучшить качество поиска по этим запросам, не ухудшая остальные запросы?

2 Метод решения

2.1 Корпус и словарные данные

Темой корпуса выбраны русскоязычные и англоязычные публикации о компьютерных играх восьми сайтов. Из MongoDB экспортирован неизменяемый локальный снимок: 27 014 документов, 85,42 МБ заголовков и основного текста в UTF-8. Из них 263 публикации участвуют только заголовком. В этой работе коллекции робота не изменяются: леммы являются производными данными для поиска. SHA-256 снимка, словаря и исполняемых файлов сохранены вместе с результатами. Экспорт обычным курсором сам по себе не является транзакционным снимком MongoDB; повторяемость измерений обеспечивается уже завершённым локальным файлом.

Для русского языка используются словоформы и парадигмы OpenCorpora из пакета `pymorphy3-dicts-ru` версии `2.4.417150.4580142`. Это ревизия 417150, скомпилированная 8 января 2022 года, а не актуальный словарь 2026 года. Исходный XML-сервер был недоступен, поэтому выбран фиксированный пакет словарных данных PyPI. Python-скрипт читает его формат через `DAWG2-Python`; готовый морфологический анализатор `pymorphy3` не используется. Нормальная форма восстанавливается из сохранённых частей парадигмы: префикса, основы и суффикса.

Для английского языка используются списки основ по частям речи и исключения WordNet 3.0. Скрипт подготовки потоково читает данные, удаляет повторные пары через временную SQLite и записывает собственный бинарный формат. Версии, контрольные суммы, сведения об источниках и лицензиях записываются в `manifest.json` и `NOTICE.txt` рядом со словарём. Это использование словарных данных, а не библиотечного алгоритма поиска.

Источники данных: пакет русских словарей, WordNet 3.0.

2.2 Собственный анализатор на C++

Токенизатор предыдущей работы сначала выделяет слова, приводит регистр и нормализует точечные сокращения. Затем лемматизатор получает нормализованный токен и возвращает одну или несколько лемм. Исходный терм, позиция и байтовый диапазон не теряются. Документ и запрос обрабатываются одинаковыми правилами.

Основные компоненты лемматизации и экспериментального поиска написаны на C++17 без контейнеров STL. Используются собственные массивы, байтовый пул и хеш-таблица с открытой адресацией и линейным пробированием. Плотность таб-

лицы не превышает 50%; хеш — FNV-1a. Словарь загружается один раз на объект, а результаты ссылаются на уже размещённые данные. Выделения памяти для каждого словарного токена не требуется. Нормально ожидаемая стоимость поиска пропорциональна длине ключа с учётом пробирований; это не гарантия постоянного времени в худшем случае. STL остаётся в разрешённом условием токенизаторе и тестовом коде, Python используется для подготовки и измерений.

Русские словоформы ищутся непосредственно в таблице. Несколько грамматических разборов с одной леммой объединяются; разные леммы сохраняются: «стали» → «сталь | стать». Контекстное снятие омонимии не выполняется. Для английских слов проверяются исключения и собственные суффиксные правила по частям речи. Полученная основа принимается только при наличии в словаре соответствующей части речи. Это не полная реализация WordNet Morphy с обработкой словосочетаний.

Неизвестное слово остаётся исходным токеном. Анализатор не переводит слова, не подбирает синонимы и не заменяет слово произвольным похожим корнем. Буквы «е» и «ё» различаются. Названия `PS5`, `C++`, `C##`, версии `3.0.1`, слова с дефисами и апострофами сохраняются после нормализации токенизатора. Например, `S.T.A.L.K.E.R.` уже поступает как `stalker`. Новые названия и сленг поэтому не исчезают из поиска.

При загрузке проверяются сигнатура формата, размеры, границы смещений, UTF-8, контрольная сумма и повторные ключи. Максимум вариантов в интерфейсе равен 64: превышение вызывает явную ошибку, а не незаметное усечение. В подготовленном словаре максимальная неоднозначность равна пяти. Файл занимает 132,99 МБ; словарь с таблицей в памяти — 166,54 МБ без буферов текущего текста. Весь корпус в RAM не загружается: обработка идёт последовательно по полям документов.

2.3 Сравнение поиска до и после

Для проверки эффекта реализована утилита `lemma_search`. Это последовательный диагностический поиск по снимку, не готовый инвертированный индекс и не TF-IDF. Она поддерживает заголовки и основной текст; в эксперименте качества используются только заголовки.

Сравниваются три режима на одинаковых документах и запросах:

- `exact`: совпадение исходных нормализованных токенов;
- `lemma`: совпадение хотя бы одной леммы каждого слова запроса;
- `hybrid`: те же кандидаты, что у `lemma`, с бонусом за точные совпадения исходных токенов.

Между различными словами запроса действует AND, между вариантами одного слова — OR. Повторные одинаковые слова запроса удаляются. Базовая оценка во всех режимах равна $h(m+1)$, где h — число покрытых в заголовке слов запроса, m — число различных слов запроса. В гибридном режиме прибавляется число точных совпадений, от 0 до m. Бонус меняет порядок только внутри группы с одинаковым покрытием заголовка. Оставшиеся равенства разрешаются по возрастанию ID документа; такой порядок детерминирован, но сам по себе не отражает релевантность.

До сравнения зафиксирован диагностический набор: 37 реальных заголовков и 12 запросов. Для каждой пары запрос–документ задана бинарная релевантность;

неразмеченных документов в этом наборе нет. Примеры специально подобраны для проверки словоформ, названий и омонимии, а не случайно выбраны из корпуса. Разметка предварительная авторская, на основе заголовков и просмотренных фрагментов; независимой экспертной оценки не проводилось. Её следует проверить перед окончательной сдачей. Поэтому полученные значения нельзя переносить на весь корпус.

Используются $P@5$, $Recall@5$ и $nDCG@5$:

$$P@5 = \frac{N_5}{5}, \qquad Recall@5 = \frac{N_5}{R},$$

где N_5 — число релевантных документов в первых пяти результатах, R — число релевантных документов именно в размеченном наборе. Если результатов меньше пяти, знаменатель точности не уменьшается.

$$DCG@5 = \sum_{i=1}^{5} \frac{2^{r_i} - 1}{\log_2(i+1)}, \qquad nDCG@5 = \frac{DCG@5}{IDCG@5}.$$

Идеальный порядок строится по известным оценкам того же набора; при нулевом идеальном значении метрика определена как ноль. Итог — среднее по запросам с одинаковыми весами. Python считает метрики по выдаче C++ и не участвует в выборе кандидатов или их ранжировании.

2.4 Тестирование и измерения

План тестирования включает словарные словоформы и омонимию, английские исключения и суффиксы, неизвестные игровые названия, защищённые токены, коллизии и повреждения бинарного формата. Проверяются время жизни ссылок на неизвестные слова, границы UTF-8 и отсутствие частичной выдачи при ошибке входного корпуса. Тесты поиска покрывают AND, повторы слов, режимы полей, стабильность top-k, ухудшение по Windows и исправление бонусом. Для расчёта метрик используются небольшие примеры с заранее вычислимым ответом.

GoogleTest и процессные проверки запускаются через CMake/CTest в Docker. Успешно выполнено 66 проверок общего C++ набора ЛР3–ЛР4 и 17 Python-тестов подготовки словаря и расчёта метрик. Журнал с отдельными тесткейсами приложен в `lab4/tests.txt`.

Производительность измерена для Windows Release-сборки GCC 13.1.0 на одном и том же локальном снимке. Для префиксов 1/8, 1/4, 1/2 и всего корпуса выполнены прогрев и 5 повторов; приводятся медианы. Время обработки включает токенизацию и поиск лемм, но не запись TSV, чтение корпуса и загрузку словаря. Последние измеряются отдельно. Измерения используют прогретый файловый кеш и не моделируют холодный запуск. Состав сайтов и языков у префиксов различается, поэтому рост времени не следует считать доказательством строгой линейности.

3 Результаты

3.1 Примеры соответствия токенов и лемм

Для наглядной проверки добавлен режим `lemmatizer demo`. Он вызывает тот же токенизатор и лемматизатор, что обработка корпуса, и выводит исходное написание, нормализованный токен, признак словарного разбора и все варианты лемм. Без

`-input` используется небольшая демонстрационная фраза; с этим параметром можно разобрать собственный UTF-8 файл.

Исходное	Токен	Лемма или леммы	Словарь
Игроки	игроки	игрок	да
играли	играли	играть	да
в	в	в	да
игры	игры	игра	да
Люди	люди	человек	да
ёлки	ёлки	ёлка	да
стали	стали	сталь | стать	да
Games	games	game	да
were	were	be	да
better	better	better | good | well	да
C++	c++	c++	нет
C#	c#	c#	нет
PS5	ps5	ps5	нет
3.0.1	3.0.1	3.0.1	нет
Джозеф	джозеф	джозеф	да
Джозефа	джозефа	джозеф	да
Джозефу	джозефу	джозеф	да
Джо	джо	джо	да

Результат реального запуска C++ с двуязычным словарём

Таблица автоматически получена из машинного TSV, а не заполнена ожидаемыми ответами вручную. Вход находится в `tests/fixtures/lab4_demo.txt`, полная таблица — в `lab4/token_demo.txt`; хеши словаря, бинарника и входа сохранены в `lab4/token_demo.json`. Позиции и байтовые границы можно проверить в `lab4/token_examples.tsv`. Отметка «нет» означает сохранение исходного токена; слово при этом не теряется.

Пример «стали» показывает неоднозначность: возвращаются «сталь» и «стать» на одной позиции. Для `better` также сохраняются несколько словарных вариантов. Это не контекстный выбор правильного значения.

Имена «Джозефа» и «Джозефу» приводятся к «джозеф», а «Джо» остаётся «джо». Здесь словарь уже содержит нужные русские формы: отсутствие перехода «Джозеф» → «Джо» не является недостатком лемматизации. Сокращённые имена относятся к синонимам и требуют отдельного расширения запроса с учётом контекста; безусловное объединение может добавить ложные совпадения. Морфологический словарь в этом эксперименте не менялся.

После добавления демонстрации 6 ноября 2026 года повторно выполнен общий набор: 66 проверок CTest и 17 Python-тестов. В C++ наборе добавлены проверки живого вывода по тестовому словарю, соответствия таблицы машинному TSV и различения словоформ имени и сокращённых имён. Измерения производительности и качества ниже относятся к прежнему зафиксированному эксперименту; этот прогон тестов не является новым замером.

Показатель	Значение
Документов	27 014
Заголовки и текст, МБ	85,42
Исходных токенов	9 020 624
Доля токенов со словарным разбором, %	80,27
Токенов с изменённой формой	3 230 212
Токенов с несколькими вариантами	1 003 108
Ключей в словаре	3 111 758

3.2 Обработка корпуса

Покрытие словарём не является точностью лемматизации: наличие разбора не гарантирует его соответствие контексту. Число исходных токенов также не равно числу вариантов лемм. На полном снимке медиана обработки составляет 6 023,36 мс, или 1 497 607 токенов/с; с чтением корпуса — 6 176,49 мс. Загрузка словаря занимает отдельно 1 046,85 мс. Для долгоживущего поискового процесса загрузка оплачивается один раз, для каждого нового запуска CLI — снова.

Документов	Текст, МБ	Обработка, мс	Загрузка, мс
3 376	7,11	307,87	1 040,60
6 753	14,90	647,41	1 048,26
13 507	42,77	2 131,37	1 071,02
27 014	85,42	6 023,36	1 046,85

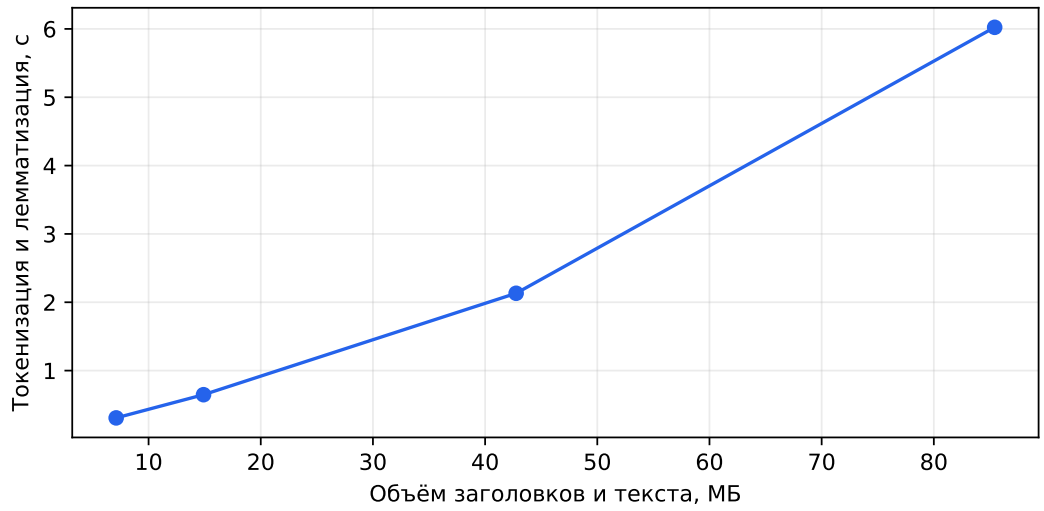


Рис. 1 — Зависимость времени обработки от объёма входного текста

Здесь КБ означает 1000 байт, МБ — 1 000 000 байт. Нельзя утверждать, что скорость оптимальна: для дальнейшей работы полезно сравнить упаковку ключей и вариантов, локальность памяти и параллельную обработку документов. Нужно учитывать и отдельную стоимость загрузки относительно короткого запроса. Повторные сырые замеры сохранены; скорость зависит от фоновой нагрузки машины.

В измеренном снимке 27 014 документов — это пока меньше требуемых курсом 30 тысяч. Словарный алгоритм не требует хранения всего корпуса в RAM, но

применимость к миллиону документов данным замером не подтверждена. Для этого нужна последующая обкачка и повторный прогон.

3.3 Качество и ошибки поиска

Режим	P@5	$Recall$@5	$nDCG$@5
Точные токены	0,217	0,292	0,282
Леммы	0,383	0,794	0,824
Леммы + точное совпадение	0,433	0,844	0,862

Средние значения на фиксированном диагностическом наборе заголовков

Запрос	Токены	Леммы	С бонусом
обновления Nintendo	0,000	1,000	1,000
модификации Minecraft	0,000	0,613	0,613
модификации Elden Ring	0,000	1,000	1,000
обзоры гарнитур	0,000	1,000	1,000
mod Skyrim	0,000	1,000	1,000
reviews Witcher	0,000	1,000	1,000
games mod	0,000	0,339	0,339
release windows	0,000	1,000	1,000
Saw	0,387	0,387	0,387
Windows	1,000	0,553	1,000
S.T.A.L.K.E.R.	1,000	1,000	1,000
Windows 10	1,000	1,000	1,000

$nDCG$@5 по запросам; оценки относятся только к выбранному набору

На запросах «обновления Nintendo», «обзоры гарнитур» и «reviews Witcher» лемматизация позволяет найти заголовки с другой словоформой. При этом она не переводит русский запрос в английский: пример с Minecraft не получает все релевантные документы только за счёт морфологии.

Важный отрицательный пример — `Windows`. Без лемматизации первые пять результатов относятся к операционной системе, $nDCG@5 = 1$. После нормализации `Windows` и `windows` сближаются с `window`. В выдачу попадают публикации про *release windows*, то есть сроки выхода игр, а не ОС. Из-за равенства базовых оценок и порядка ID значение падает примерно до 0,553. Бонус за исходное точное совпадение возвращает пять заголовков про Windows и значение 1. На остальных запросах этого набора ухудшения от бонуса не наблюдалось; это не гарантия для всех запросов.

В запросе `Windows 10` дополнительный контекст уже отсекает эти ложные кандидаты. У `Saw` остаются другие трудности: название франшизы совпадает с обычным английским словом, а притяжательная форма с апострофом сохраняется токенизатором и может не совпасть с запросом. Один бонус не устраняет смысловую неоднозначность. Возможные дальнейшие улучшения — контекст, учёт названий и явные правила притяжательных форм с отдельной проверкой качества. Словарное расширение всех вариантов повышает полноту, но способно снизить точность.

Разметка, полные списки выдачи и метрики находятся в `tests/fixtures/lab4_judgments.json`, `lab4/quality.json` и локальном каталоге эксперимента. Демонстрация показывает сохранённый эксперимент, а не выдаёт его за новый поиск. Положительный результат на 37 заголовках — иллюстрация механизма, не репрезентативная оценка поисковой системы на всех 27 014 публикациях.

3.4 Исходный код и воспроизводимость

Инструкция подготовки словаря, сборки, запуска автотестов и эксперимента приведена в `doc/lab4_run.md`. Ветке этой работы соответствует адрес github.com/kirbasss/MAI-IR/tree/lab4. QR-код ведёт на тот же адрес; доступность последних изменений зависит от синхронизации локальной ветки с GitHub.

Рис. 2 — Ссылка на ветку лабораторной работы №4

4 Вывод

В этой работе я добавил словарную лемматизацию русских и английских слов и проверил, как она меняет поиск на реальных игровых публикациях. Было интересно разобраться, чем словарь словоформ отличается от готового анализатора и почему нельзя всегда выбирать единственную нормальную форму. Собственная хеш-таблица позволила обрабатывать текст без выделения памяти для каждого найденного слова.

На небольшом размеченном наборе поиск стал находить больше нужных заголовков, но пример с Windows показал и обратную сторону: объединение словоформ иногда добавляет посторонние результаты. Сохранение исходного слова и бонус за точное совпадение помогли в этом примере. Пока разметку стоит перепроверить, а корпус увеличить хотя бы до минимального размера из требований курса. В дальнейшем этот модуль можно использовать при построении индекса, не меняя уже скачанные документы и правила токенизации.